

Documentation

Prérequis

Une base de donnée mariadb opérationnel

Installation et préparation

1. Cloner le dépôt

Commence par récupérer le projet depuis GitHub :

```
git clone https://github.com/kaelianbaudelet/projet-cybersecurite-AP1.git
```

2. Accéder au répertoire du projet

Une fois le dépôt cloné, rends-toi dans le dossier correspondant :

```
cd projet-cybersecurite-AP1/dbbackup-cli
```

3. Rendre le script exécutable

Avant de pouvoir utiliser la CLI, il faut donner les droits d'exécution au script principal :

```
chmod +x ./dbbackup
```

4. Lancer la CLI

Vous pouvez maintenant exécuter le script principal directement à l'aide de la commande suivante :

```
./dbbackup [ACTION]
```

Cette commande permet d'utiliser la CLI pour exécuter les différentes actions disponibles (sauvegarde, restauration, transfert, etc.) décrites dans la section **Liste des principales commandes du CLI**. Elle sert notamment à effectuer des sauvegardes de bases de données de manière simple et automatisée.

NOTE : Si vous le souhaitez, vous pouvez créer un alias pour exécuter la CLI plus facilement :

- **Sur Linux**, vous pouvez placer le script dbbackup dans un dossier présent dans votre PATH (par exemple /usr/local/bin/) afin de pouvoir l'appeler simplement avec la commande dbbackup depuis n'importe quel répertoire.
- **Sur macOS**, vous pouvez également ajouter un alias dans votre fichier ~/.zshrc ou ~/.bashrc, par exemple :

```
alias dbbackup="/chemin/vers/le/projet/dbbackup-cli/dbbackup"
```

- Après avoir sauvegardé le fichier, rechargez le terminal avec :

```
source ~/.zshrc
```

Vous pourrez alors utiliser la CLI en tapant simplement : dbbackup [commande]

Liste des principales commandes du CLI

`dbbackup backup`

Créer une sauvegarde de base de données, compressée, chiffrée et optionnellement transférée vers un serveur distant.

Options utiles :

- `-h|--host` hôte MariaDB (default localhost).
- `-P|--port` port MariaDB (default 3306).
- `-u|--user` utilisateur MariaDB (obligatoire).
- `-p|--password` mot de passe (obligatoire).
- `-d|--database` base à sauvegarder (obligatoire).
- `-o|--output` dossier local (default backups/).
- `--encrypt` / `--no-encrypt` active ou désactive le chiffrement (default actif).
- `--transfer` / `--no-transfer` force ou interdit l'envoi distant.
- `--remote <nom>` choisit un serveur distant et active automatiquement le transfert (utilise `config/remotes/<nom>.conf`).

Exemples :

Sauvegarde locale simple

```
./dbbackup backup -u root -p secret -d mydb
```

Sauvegarde sans chiffrement

```
./dbbackup backup -u admin -p pass -d production --no-encrypt
```

Sauvegarde et envoi vers le remote "prod"

```
./dbbackup backup -u root -p secret -d mydb --remote prod
```

`dbbackup backup list`

Liste les sauvegardes locales ou distantes disponibles.

Options :

- `-o|--output <dossier>` analyse un dossier spécifique.

- `--remote` (optionnellement `--remote=<nom>`) affiche les sauvegardes sur un serveur distant. Sans nom, le serveur par défaut est utilisé.

Exemples :

Lister les fichiers locaux

```
./dbbackup backup list
```

Lister les sauvegardes distantes depuis "prod"

```
./dbbackup backup list --remote=prod
```

dbbackup restore

Restaure une base à partir d'une sauvegarde locale ou distante. Le script décompresse, déchiffre et valide la somme de contrôle avant d'utiliser mysql.

Options :

- `-f|--file <chemin>` fichier de sauvegarde (obligatoire sauf `--list`).
- `-h|--host`, `-P|--port`, `-u|--user`, `-p|--password`, `-d|--database` paramètrent la base cible.
- `--remote` télécharge la sauvegarde depuis le serveur distant actif (utilise le nom passé à `-f` relatif au répertoire distant).
- `--verify` vérifie l'intégrité sans restaurer.
- `--list` affiche les sauvegardes locales disponibles.

Le script vérifie l'existence de la base, peut la créer sur demande, et requiert la confirmation `yes` avant d'écraser une base existante.

Exemples :

Restaurer une sauvegarde locale chiffrée

```
./dbbackup restore -f backups/mydb_20241110_020000.sql.gz.enc -u root -p secret -d mydb
```

Vérifier uniquement la sauvegarde

```
./dbbackup restore -f backups/mydb_20241110_020000.sql.gz.enc --verify
```

Restaurer un fichier présent sur le serveur distant "prod"

```
./dbbackup restore -f daily/mydb-2024-11-10.sql.gz.enc --remote -u root -p secret -d mydb
```

dbbackup schedule

Gère les sauvegardes planifiées via cron. Toutes les planifications sont stockées dans config/schedules.conf.

Sous-commandes :

- `schedule add` : ajoute une planification. Nécessite `-n|--name`, `-c|--cron`, `-u|--user`, `-p|--password`, `-d|--database`. `--remote <nom>` rend la planification distante.
- `schedule list` : affiche un tableau des planifications configurées et leur statut.
- `schedule modify <nom>` : met à jour cron, hôte, identifiants ou serveur distant (`--remote` ou `--no-remote`).
- `schedule remove <nom>` : supprime la planification et la tâche cron associée.
- `schedule enable <nom>` / `schedule disable <nom>` : active ou désactive la planification tout en installant/retirant la tâche cron.
- `schedule next` : rappelle la prochaine exécution théorique pour chaque planification active.

Exemples :

Planifier une sauvegarde quotidienne locale à 02h00

```
./dbbackup schedule add -n daily -c "0 2 * * *" -u root -p secret -d mydb
```

Ajouter une sauvegarde distante

```
./dbbackup schedule add -n daily-remote -c "0 3 * * *" -u root -p secret -d mydb  
--remote prod
```

Modifier une planification pour retirer le transfert distant

```
./dbbackup schedule modify daily-remote --no-remote
```

Suspendre puis réactiver une planification

```
./dbbackup schedule disable daily  
./dbbackup schedule enable daily
```

dbbackup remote

Administre les serveurs distants utilisés pour les transferts et restaurations. Chaque serveur correspond à un fichier config/remotes/<nom>.conf.

Sous-commandes :

- `remote list` : liste les serveurs connus (le serveur par défaut est marqué d'une *).

- `remote add <nom> --host <hote> --user <utilisateur> [options]` : cree un nouveau profil.
Options disponibles : `--port`, `--path`, `--auth key|password`, `--ssh-key`, `--password`, `--verify yes|no`, `--delete-after yes|no`, `--set-default`.
- `remote show <nom>` : affiche la configuration detaillee.
- `remote remove <nom>` : supprime le profil (et reinitialise le defaut si besoin).
- `remote set-default <nom>` : definit le serveur utilise par defaut quand `--remote n` est pas renseigne.
- `remote test <nom>` : tente une connexion SSH/SFTP avec les parametres stockes.

Exemples :

Ajouter un serveur distant base sur une cle SSH

```
./dbbackup remote add prod --host backup.example.com --user backup --auth key --ssh-key  
~/.ssh/id_rsa --path /srv/backups --set-default
```

Afficher la configuration puis tester la connexion

```
./dbbackup remote show prod  
./dbbackup remote test prod
```

`dbbackup config show`

Affiche le contenu de `config/dbbackup.conf`, le résumé des transferts (`config/transfer.conf`), la liste des serveurs distants disponibles et les chemins importants (clé de chiffrement, fichier de planification). Utile pour vérifier rapidement la configuration active.

Exemple :

```
./dbbackup config show
```